

PRAKTIKUM SISTEM OPERASI

MODUL 6

(Sekuensial dan Konkuren)



**Universitas
Telkom**

Oleh:

(Haza Zaidan Zidna Fann) (2311104056)

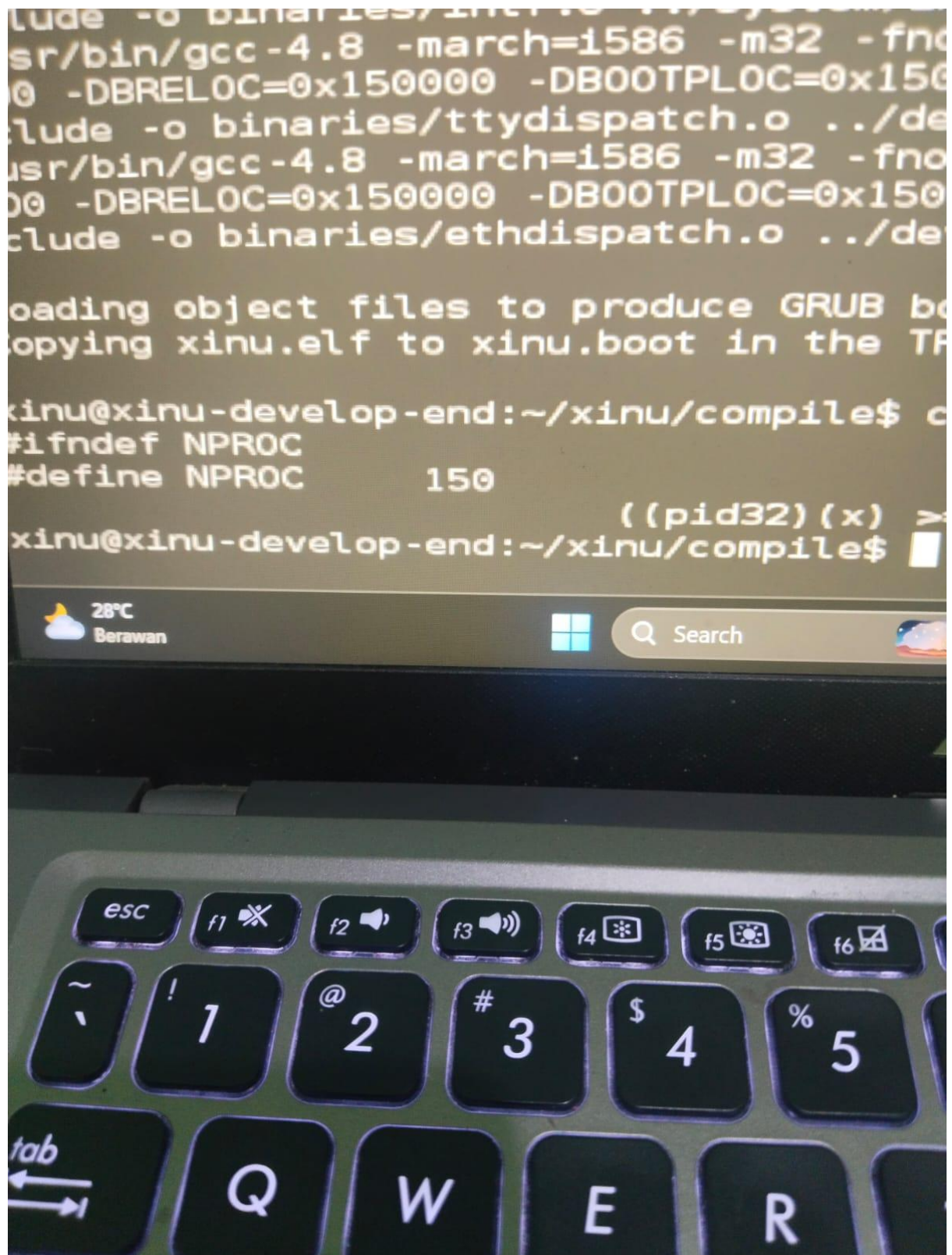
PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

1.



Pada percobaan ini dilakukan perubahan nilai **NPROC** pada file `process.h` dari 8 menjadi 150 untuk meningkatkan jumlah maksimum proses yang dapat dijalankan secara bersamaan pada sistem operasi Xinu. Langkah ini dilakukan dengan mengedit file pada direktori `~/xinu/include`, kemudian melakukan kompilasi ulang agar perubahan diterapkan. Tujuan perubahan ini adalah untuk

mendukung percobaan multitasking pada tahap selanjutnya, sehingga sistem mampu menangani lebih banyak proses tanpa mengalami keterbatasan seperti sebelumnya.

2.

```
xinu@xinu-dev
File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ sudo minicom
[sudo] password for xinu:
Sorry, try again.
[sudo] password for xinu:

Xinu for Vbox -- version #113 (xinu) Mon 13
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9a:f4:08
4447448 bytes of free memory. Free list:
[0x0015E320 to 0x0009FFFF]
[0x00100000 to 0x005FBFFF]
115177 bytes of Xinu code.
[0x00100000 to 0x0011C1E8]
148488 bytes of data.
[0x0011FEC0 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

28°C
Barawon

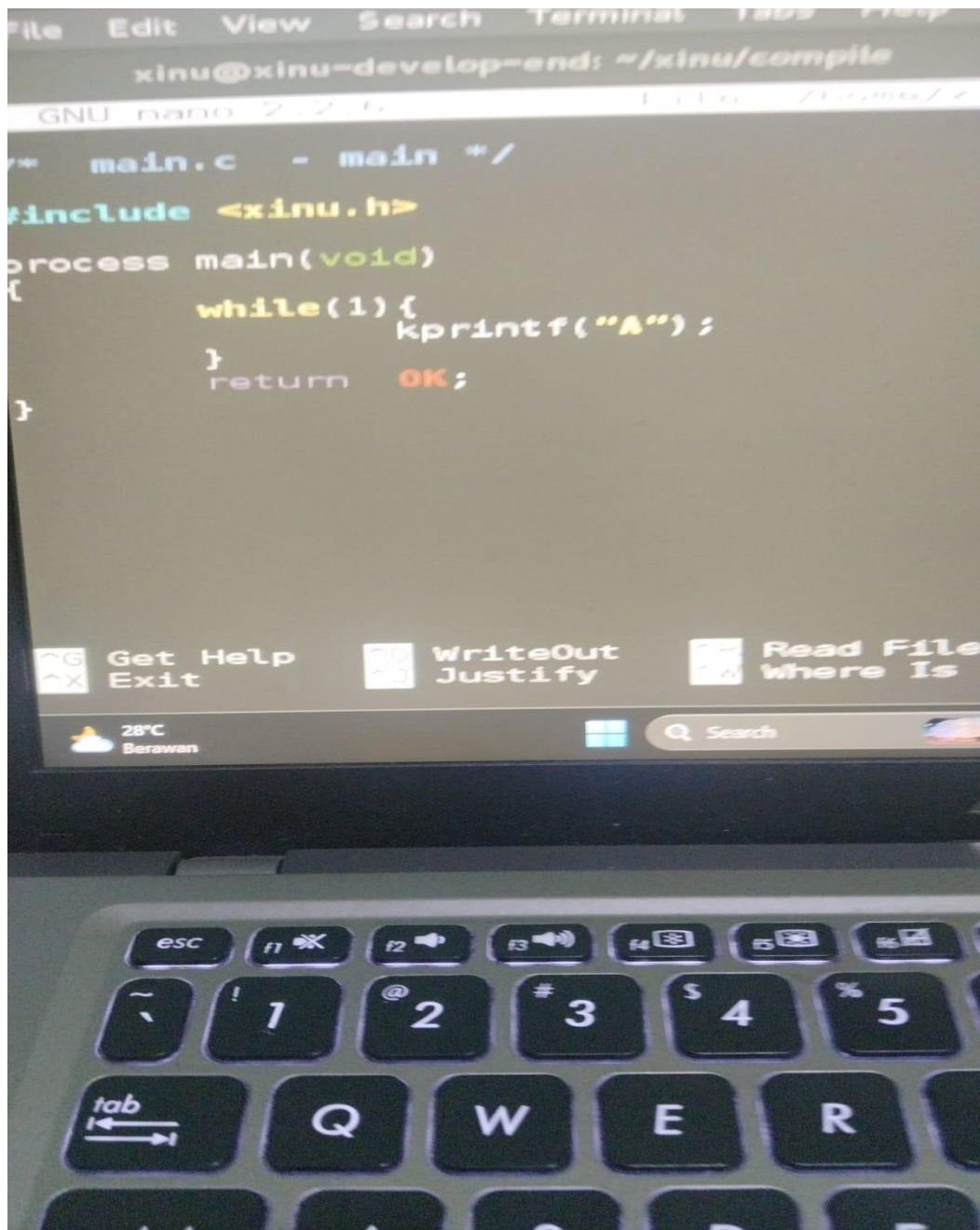


Search



ASUS





Output pada layar tersebut menunjukkan bahwa sistem operasi Xinu berhasil dijalankan di lingkungan virtual (melalui QEMU/VirtualBox) dan sudah masuk ke tahap inisialisasi kernel dengan baik, ditandai dengan munculnya informasi versi, deteksi perangkat keras seperti NIC (Intel Ethernet), alamat MAC, serta jumlah memori yang tersedia beserta pembagian area kode dan data di RAM; kemudian sistem juga berhasil memperoleh alamat IP (10.0.3.15) yang menandakan modul jaringan aktif dan berfungsi, dan deretan huruf "AAAAAAA..." yang muncul terus-menerus biasanya berasal dari program uji (test program) atau proses di dalam Xinu yang sengaja mencetak output ke serial sebagai indikator bahwa sistem berjalan secara kontinu tanpa crash, sehingga secara keseluruhan ini menandakan bahwa proses kompilasi, booting kernel, serta komunikasi melalui serial (minicom) sudah berhasil dan sistem Xinu dalam kondisi running normal.

3.

4447448 bytes of free memory. Fr
[0x0015E320 to 0x0009FFF7
[0x00100000 to 0x005FBFFF
115327 bytes of Xinu code.
[0x00100000 to 0x0011C27E]
148456 bytes of data.
[0x0011FEE0 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a

A
B
A
B
A
B
A
B
A
B
A
B
A
B

CTRL-A Z for help | 115200 8N1 | NOR | MI


```
#include <xinu.h>
```

```
int k = 0;
```

```
void buat() {
```

```
    while (1) {
```

```
        k = k + 1;
```

```
        kprintf("Buat: %d\n", k);
```

```
        sleep(1);
```

```
    }
```

```
}
```

```
void lepas() {
```

```
    while (1) {
```

```
        k = k - 1;
```

```
        kprintf("Lepas: %d\n", k);
```

```
        sleep(1);
```

```
    }
```

```
}
```

```
process main(void) {
```

```
    resume(create(buat, 1024, 20, "buat", 0));
```

```
    resume(create(lepas, 1024, 20, "lepas", 0));
```

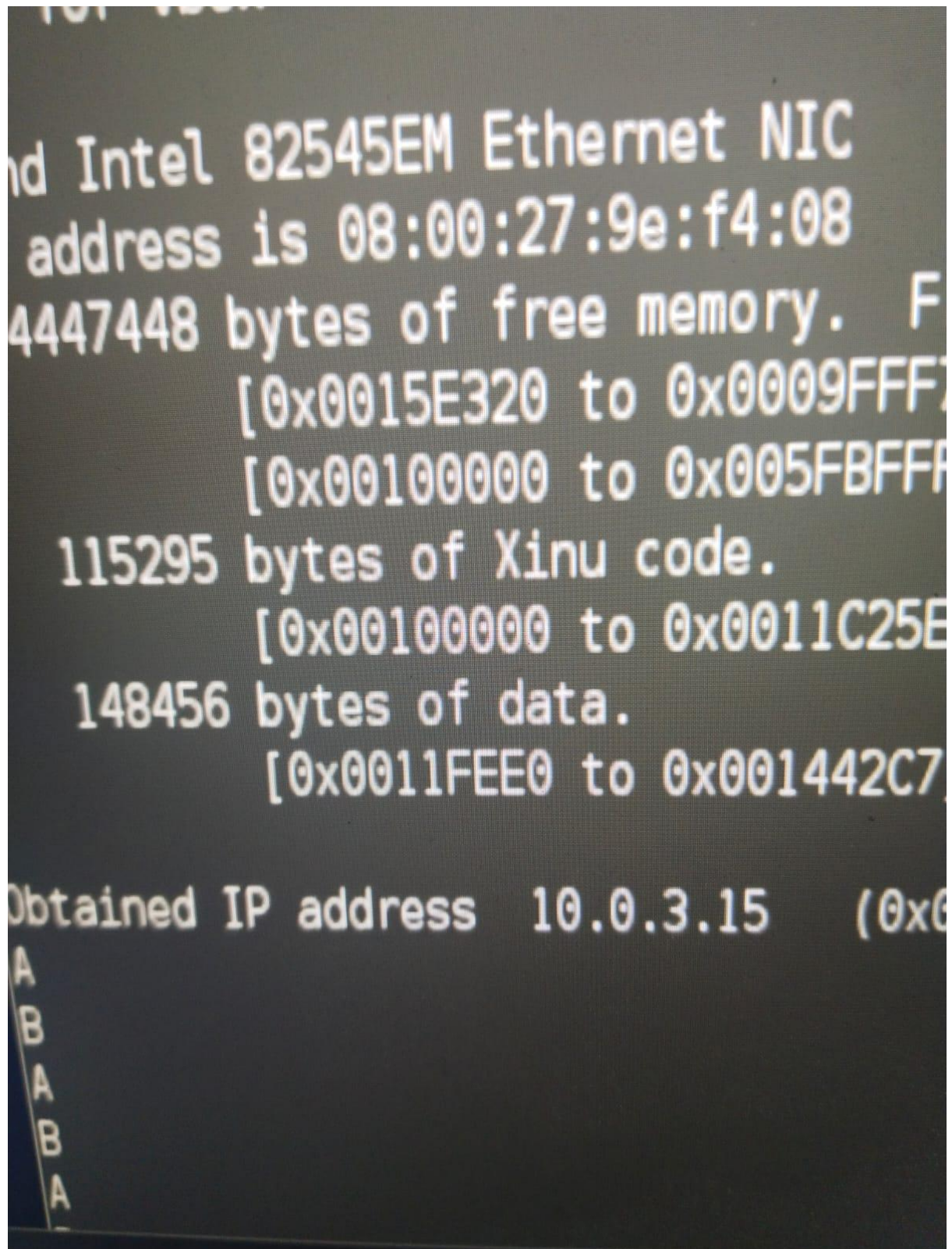
```
    return OK;
```

```
}
```

proses booting sistem operasi **Xinu** yang dijalankan di lingkungan virtual seperti VirtualBox, di mana sistem pertama kali mendeteksi perangkat keras seperti kartu jaringan (Intel 82545EM Ethernet NIC) beserta alamat MAC-nya, kemudian menampilkan informasi memori yang tersedia termasuk alamat rentang memori bebas, ukuran kode program (Xinu code), serta data yang digunakan sistem; setelah itu sistem berhasil memperoleh alamat IP (10.0.3.15) yang menandakan koneksi jaringan sudah aktif, dan bagian bawah yang menampilkan huruf “A” dan “B” secara bergantian merupakan hasil dari implementasi proses konkuren (multitasking) menggunakan fungsi seperti create() dan resume() di dalam kode, di mana dua proses berbeda dijalankan secara bersamaan—satu mencetak “A” dan satu lagi mencetak “B”—sehingga output terlihat saling bergantian akibat scheduler Xinu

yang mengatur eksekusi CPU antar proses, sementara baris paling bawah (status bar Minicom) menunjukkan konfigurasi komunikasi serial seperti baud rate (115200), mode (8N1), serta tipe terminal (VT102), yang digunakan untuk menampilkan output sistem Xinu secara real-time melalui emulator terminal.

4.



The image shows a screenshot of a Minicom terminal window with a dark background and light-colored text. The text displays the output of the Xinu system boot process. It includes information about the Intel 82545EM Ethernet NIC, memory status (4447448 bytes of free memory), Xinu code size (115295 bytes), and data size (148456 bytes). It also shows the obtained IP address 10.0.3.15. On the left side, there is a vertical status bar with the letters 'A', 'B', 'A', 'B', 'A' stacked vertically.

```
nd Intel 82545EM Ethernet NIC  
address is 08:00:27:9e:f4:08  
4447448 bytes of free memory. F  
[0x0015E320 to 0x0009FFF7  
[0x00100000 to 0x005FBFFF  
115295 bytes of Xinu code.  
[0x00100000 to 0x0011C25E  
148456 bytes of data.  
[0x0011FEE0 to 0x001442C7  
Obtained IP address 10.0.3.15 (0x0  
A  
B  
A  
B  
A
```

```

/* main.c - main */
#include <xinu.h>

void sndChar(char);

process main(void)
{
    resume(create(sndChar, 1024, 20, "send A", 1, 'A'));
    resume(create(sndChar, 1024, 20, "send B", 1, 'B'));
    return OK;
}

void sndChar(char c)
{
    while (1) {
        kprintf("%c\n", c);
        sleepms(1000);
    }
}

```

Pada percobaan nomor 4, program dikembangkan lebih lanjut dari konsep sebelumnya dengan menambahkan mekanisme pengaturan atau sinkronisasi terhadap dua proses yang berjalan secara bersamaan, di mana proses seperti buat dan lepas tidak hanya sekadar mencetak output secara bergantian, tetapi juga mulai dikontrol agar tidak saling mengganggu atau menghasilkan kondisi tidak konsisten (race condition) pada variabel bersama; hal ini biasanya dilakukan dengan memanfaatkan fitur sinkronisasi di Xinu seperti semaphore atau pengaturan delay (sleep) untuk mengatur giliran eksekusi, sehingga ketika satu proses sedang mengakses atau memodifikasi variabel, proses lainnya harus menunggu hingga proses tersebut selesai, yang pada akhirnya menghasilkan output yang lebih terstruktur dan tidak acak, sekaligus menunjukkan bagaimana sistem operasi mengelola konkurensi, pembagian CPU, serta koordinasi antar proses secara lebih aman dan terkontrol dibandingkan hanya menjalankan proses secara paralel tanpa pengaturan seperti pada percobaan sebelumnya.

xinu@xinu-dev

File Edit View Search Terminal Help

ilai dari n adalah 1000
ilai dari n adalah 1000
ilai dari n adalah 1000
ilai dari n adalah 1000
ilai dari n adalah 1000
ilai dari n adalah 1000
ilai dari n adalah 1000

```
/* main.c - main */
```

```
#include <xinu.h>
```

```
int32 n = 0;
```

```
void produser(void);
```

```
void konsumer(void);
```

```
process main(void)
```

```
{
```

```
    resume(create(produser, 1024, 20, "produser", 0));
```

```
    resume(create(konsumer, 1024, 20, "konsumer", 0));
```

```
    return OK;
```

```
}
```

```
void produser(void)
```

```

{
    int32 i;

    for (i = 1; i <= 1000; i++) {
        n++;
    }
}

void konsumer(void)
{
    int32 i;

    for (i = 1; i <= 1000; i++) {
        kprintf("Nilai dari n adalah %d\n", n);
    }
}

```

producer–consumer dikembangkan lebih lanjut dengan menambahkan mekanisme sinkronisasi menggunakan semaphore untuk menghindari terjadinya race condition pada variabel bersama, di mana proses produser dan konsumer tidak lagi berjalan bebas seperti pada percobaan sebelumnya, tetapi diatur agar tidak mengakses variabel secara bersamaan yang dapat menyebabkan inkonsistensi data; semaphore berfungsi sebagai alat pengunci (locking) yang memastikan hanya satu proses yang dapat mengakses critical section dalam satu waktu, sehingga ketika produser sedang menambah nilai variabel, konsumer harus menunggu hingga produser selesai, dan sebaliknya, hal ini membuat alur eksekusi menjadi lebih terkontrol, aman, dan sesuai dengan prinsip manajemen proses pada sistem operasi, sekaligus menunjukkan bagaimana Xinu menangani koordinasi antar proses dalam lingkungan multitasking untuk menjaga integritas data dan mencegah konflik akses memori.